

BinX TECH • AI & MACHINE LEARNING INTERNSHIP PROGRAM

WEEK 1

PYTHON & DATA SCIENCE FOUNDATIONS

Python Environment, NumPy, Pandas, Matplotlib & the Jupyter Workflow

The first week of Phase 1: setting up a professional Python data-science environment and building fluency in the four libraries every AI/ML workflow is built on — NumPy for numerical arrays, Pandas for tabular data, Matplotlib for visualization, all inside the Jupyter Notebook workflow.

PHASE 1 • 40 HOURS • 5 TRAINING DAYS • HANDS-ON

Prepared by BinX Tech
Palestine | Nablus

BinX
TECH

1. Week 1 Overview

Week	Week 1 of 10 — Phase 1: Foundations
Total Hours	40 hours (full-time track) / 20 hours (part-time track, Weeks 1-2 combined)
Format	On-site / Remote / Hybrid — all work in Jupyter notebooks committed to GitHub
Focus	Python environment setup, NumPy arrays, Pandas DataFrames, Matplotlib plotting, Jupyter Notebook workflow
Mentor Supervision	Daily check-in with assigned ML mentor; environment verified before hands-on work begins

2. Week 1 Learning Objectives

- Set up a professional, reproducible Python data-science environment (venv/conda, pip, Jupyter) reused for the entire 400-hour program.
- Write clean, idiomatic Python: data types, control flow, functions, list comprehensions, and basic OOP.
- Create, index, slice, and vectorize computations over NumPy arrays instead of using slow Python loops.
- Load, inspect, clean, filter, and aggregate tabular data with Pandas DataFrames.
- Produce clear line, scatter, bar, and histogram plots with Matplotlib to explore and communicate data.
- Work fluently in the Jupyter Notebook environment and commit a well-structured notebook to GitHub.

3. Daily Schedule

Day	Hours	Topic Focus
Day 1	8 hrs	Environment setup (Python, venv/conda, pip, Jupyter, VS Code, Git); Jupyter workflow
Day 2	8 hrs	Python for data science: data types, control flow, functions, list comprehensions, OOP basics
Day 3	8 hrs	NumPy: array creation, indexing/slicing, broadcasting, vectorized operations
Day 4	8 hrs	Pandas: Series & DataFrames, loading data, inspection, selection, cleaning, groupby
Day 5	8 hrs	Matplotlib: line/scatter/bar/histogram plots, subplots, labeling; Week 1 mini-notebook

4. Day-by-Day Curriculum

Day 1 — Environment Setup & the Jupyter Workflow — 8 hours

Learning Objectives

- Set up a reproducible Python environment with a virtual environment and pip.
- Work fluently in the Jupyter Notebook cell-based workflow (code + Markdown).
- Initialize a Git repository and make a first commit of a working notebook.

Key Topics

- Why a reproducible environment matters for all AI/ML work
- Virtual environments (venv/conda), pip, and requirements.txt
- The Jupyter Notebook workflow: code cells vs. Markdown cells
- Supporting tools: VS Code, Git/GitHub, Google Colab

Lesson Content

1.1 Why the Environment Comes First

Every AI/ML task in this program runs on top of a Python environment. Setting it up correctly and reproducibly on Day 1 — so the exact same libraries and versions can be recreated later — is what prevents the most common and frustrating class of problems: code that works on one machine but not another. This is the same reproducibility discipline that MLflow and requirements.txt enforce later in the program.

1.2 Python, pip, and Virtual Environments

Python is the primary language for all AI/ML work in this program. A virtual environment is an isolated, per-project Python installation, so that each project's dependencies never conflict with another's. pip installs packages into the active environment. Isolating every project is standard professional practice, not optional.

```
$ python3 -m venv .venv          # create an isolated environment
$ source .venv/bin/activate      # activate it (Linux/macOS)
$ pip install numpy pandas matplotlib jupyter
$ pip freeze > requirements.txt  # record exact versions for reproducibility
```

conda is a common alternative that manages both Python versions and packages together, widely used in data science; either venv+pip or conda is acceptable for this program as long as the environment is reproducible.

1.3 The Jupyter Notebook Workflow

Jupyter Notebook is the primary interactive environment for AI/ML experimentation. A notebook is a sequence of cells that are either code (executed in place, with output shown directly below) or Markdown (formatted text used to document what each step does and why). This mix of live code, output, and narrative is exactly what makes a notebook the standard deliverable for data-science work — it tells a complete, reproducible story.

Cell Type	Purpose
Code	Runs Python and displays its output (numbers, tables, plots) directly beneath
Markdown	Formatted narrative: explains the goal, the approach, and the interpretation of results

A professional notebook reads top-to-bottom like a report: Markdown explains each step, code executes it, and outputs are shown — never a wall of undocumented code cells.

1.4 Supporting Tools: VS Code, Git & Colab

VS Code is the IDE for production-grade Python scripts (with the Python, Pylance, and Jupyter extensions). Git and GitHub provide version control — every week's notebooks are committed to a repository, building the portfolio that employers actually look at. Google Colab offers free GPU/TPU access for the deep-learning weeks later in the program, running the same notebook format in the browser.

Hands-On Lab: Build the Data-Science Environment

- Step 1: Create and activate a virtual environment, then install numpy, pandas, matplotlib, and jupyter.
- Step 2: Launch Jupyter and create a notebook mixing at least one Markdown cell and one code cell.
- Step 3: Run a code cell that prints the installed version of each library to confirm the setup.
- Step 4: Freeze the environment to requirements.txt.
- Step 5: Initialize a Git repository, commit the notebook and requirements.txt, and push to GitHub.

Tools Used

Python 3.10+ • venv / conda • pip • Jupyter Notebook • VS Code • Git & GitHub

Day 2 — Python for Data Science — 8 hours

Learning Objectives

- Use Python's core data types and control flow fluently.
- Write clean, reusable functions with clear parameters.
- Write list comprehensions and read basic object-oriented code.

Key Topics

- Core data types: int, float, str, bool, list, tuple, dict, set
- Control flow: if/elif/else, for, while
- Functions: parameters, return values, docstrings
- List comprehensions
- Object-oriented programming basics: classes, attributes, methods

Lesson Content

2.1 Core Python Data Types

Before any data science, you need fluency in Python's building blocks. The core types you will use constantly:

Type	Description	Example
int / float	Whole numbers and decimals	42, 3.14
str	Text	"churn"
bool	True / False	True
list	Ordered, mutable sequence	[1, 2, 3]
tuple	Ordered, immutable sequence	(x, y)
dict	Key-value pairs	{"age": 30}
set	Unordered unique values	{1, 2, 3}

2.2 Control Flow

Conditionals (if/elif/else) branch logic; loops (for, while) repeat work. In data science you will loop far less than a general programmer — vectorized NumPy/Pandas operations (covered Days 3-4) replace most loops — but understanding control flow is still essential for data cleaning logic and custom functions.

```
for value in [1, 2, 3, 4]:
    if value % 2 == 0:
        print(f"{value} is even")
    else:
        print(f"{value} is odd")
```

2.3 Functions

Functions package reusable logic. A well-named function with clear parameters and a single responsibility is the foundation of clean, testable data pipelines.

```
def normalize(value, min_val, max_val):
    """Scale a value to the 0-1 range."""
    return (value - min_val) / (max_val - min_val)

print(normalize(75, 0, 100))    # 0.75
```

2.4 List Comprehensions

A list comprehension builds a new list in one readable line, replacing a multi-line loop. This is idiomatic ("Pythonic") code and appears constantly in real data-science work.

```
squares = [x**2 for x in range(10)]
evens   = [x for x in range(20) if x % 2 == 0]
```

2.5 Object-Oriented Programming (Basics)

A class bundles data (attributes) and behavior (methods) together. You will not write many classes early on, but AI/ML frameworks (Scikit-learn estimators, PyTorch models) are all built on classes, so understanding the syntax is essential for reading and using them later.

```
class DataPoint:
    def __init__(self, x, y):
        self.x = x
        self.y = y
    def distance_from_origin(self):
        return (self.x**2 + self.y**2) ** 0.5

p = DataPoint(3, 4)
print(p.distance_from_origin())    # 5.0
```

Hands-On Lab: Python Fundamentals Drill

- Step 1: Write a function that takes a list of numbers and returns the mean, min, and max as a dictionary.
- Step 2: Rewrite a given for-loop that filters even numbers as a single list comprehension.
- Step 3: Define a small class representing a data record with at least two attributes and one method.
- Step 4: Document each cell with a Markdown explanation of what it does and why.

Tools Used

Python 3.10+ • Jupyter Notebook

Day 3 — NumPy: Numerical Computing — 8 hours

Learning Objectives

- Create NumPy arrays in multiple ways and inspect their shape and dtype.

- Index and slice 1D and 2D arrays, including boolean masking.
- Use vectorized operations and broadcasting instead of Python loops.

Key Topics

- Why NumPy underpins the entire AI/ML stack
- Array creation: array, zeros, ones, arange, linspace, random
- Attributes, indexing, and slicing (including 2D)
- Vectorized operations and boolean masking
- Broadcasting

Lesson Content

3.1 Why NumPy

NumPy is the numerical foundation that Pandas, Scikit-learn, TensorFlow, and PyTorch are all built on. Its core object is the ndarray — a fixed-size, homogeneous (all-same-type) array that stores data far more compactly and processes it far faster than a Python list, because operations run in optimized C code rather than a Python loop.

3.2 Creating Arrays

```
import numpy as np
a = np.array([1, 2, 3, 4])      # from a list
z = np.zeros((2, 3))          # 2x3 array of zeros
o = np.ones(5)                 # 1D array of ones
r = np.arange(0, 10, 2)        # [0 2 4 6 8]
l = np.linspace(0, 1, 5)       # 5 evenly spaced values 0..1
rnd = np.random.rand(3, 3)     # 3x3 random values in [0,1)
```

3.3 Array Attributes, Indexing & Slicing

Every array has a shape (dimensions), dtype (data type), and ndim (number of dimensions). Indexing and slicing work like lists but extend to multiple dimensions with a comma: array[row, column].

```
m = np.array([[1, 2, 3], [4, 5, 6]])
print(m.shape)      # (2, 3)
print(m[0, 2])      # 3 (row 0, column 2)
print(m[:, 1])      # [2 5] (all rows, column 1)
print(m[1, :])      # [4 5 6] (row 1, all columns)
```

3.4 Vectorized Operations

The core reason to use NumPy: operations apply to the whole array at once, with no explicit loop. This is called vectorization, and it is both dramatically faster and far more readable than looping element by element.

```
a = np.array([1, 2, 3, 4])
print(a * 2)        # [2 4 6 8]
print(a + a)        # [2 4 6 8]
print(a.mean())     # 2.5
print(a[a > 2])     # [3 4] (boolean masking)
```

3.5 Broadcasting

Broadcasting is the rule that lets NumPy combine arrays of different but compatible shapes without copying data — for example, adding a single number to every element, or adding a 1D row to every row of a 2D array. Understanding broadcasting is what separates fluent NumPy use from constant shape-mismatch errors.

```
matrix = np.array([[1, 2, 3], [4, 5, 6]])
row     = np.array([10, 20, 30])
print(matrix + row)  # row is added to EACH row of the matrix
```

Hands-On Lab: NumPy Array Manipulation

- Step 1: Create a 2D array of shape (4, 4) filled with values 1-16 and print its shape and dtype.
- Step 2: Use slicing to extract the second column and the last row.
- Step 3: Use a boolean mask to select all values greater than the array's mean.
- Step 4: Add a 1D row array to every row of a 2D array using broadcasting, and verify the result manually.

Tools Used

NumPy • Jupyter Notebook

Day 4 — Pandas: Tabular Data — 8 hours

Learning Objectives

- Load a real dataset into a DataFrame and inspect its structure.
- Select columns and filter rows by condition.
- Clean missing values and duplicates, and aggregate data with groupby.

Key Topics

- Series vs. DataFrame
- Loading and inspecting data: read_csv, head, info, describe
- Selecting and filtering: columns, boolean filtering, .loc/.iloc
- Cleaning: missing values, duplicates, type conversion
- Grouping and aggregation: groupby, agg (split-apply-combine)

Lesson Content

4.1 Series and DataFrames

Pandas is the primary tool for working with tabular (spreadsheet-like) data. Its two core objects: a Series is a single labeled column, and a DataFrame is a full table — rows and named columns — which is the format almost all real-world datasets arrive in (CSV, Excel, SQL query results).

4.2 Loading and Inspecting Data

The first step with any dataset is always to load it and look at it before doing anything else — its shape, its columns, its data types, and a sample of rows.

```
import pandas as pd
df = pd.read_csv("data.csv")
df.head()          # first 5 rows
df.shape           # (rows, columns)
df.info()          # column names, non-null counts, dtypes
df.describe()      # summary statistics for numeric columns
```

4.3 Selecting and Filtering

Select columns by name, and rows by condition (boolean filtering) — the Pandas equivalent of NumPy's boolean masking. .loc selects by label, .iloc by integer position.

```
df["age"]          # a single column (Series)
df[["age", "income"]] # multiple columns (DataFrame)
df[df["age"] > 30]   # rows where age > 30
df.loc[df["age"] > 30, "income"] # income column for those rows
```

4.4 Cleaning Data

Real data is messy: missing values, duplicates, and wrong types are the norm. Cleaning is often the largest part of any ML project, and doing it correctly is what everything downstream depends on.


```
df.isnull().sum()          # count missing values per column
df["age"].fillna(df["age"].mean(), inplace=True) # fill missing with mean
df.dropna(inplace=True)    # drop rows with any missing value
df.drop_duplicates(inplace=True)
```

4.5 Grouping and Aggregation

groupby splits the data into groups by a column's values, applies an aggregation (mean, sum, count) to each group, and combines the results — the single most useful Pandas operation for finding patterns in data ("split-apply-combine").

```
df.groupby("category")["income"].mean()
df.groupby("category").agg({"income": "mean", "age": "max"})
```

Hands-On Lab: Exploring a Real Dataset with Pandas

- Step 1: Load a provided CSV dataset into a DataFrame and report its shape, columns, and dtypes.
- Step 2: Count missing values per column and handle them (fill or drop) with a documented justification.
- Step 3: Filter the data to a meaningful subset (e.g. rows above a threshold) and describe what you find.
- Step 4: Use groupby to compute an aggregate statistic per category and interpret the result in a Markdown cell.

Tools Used

Pandas • Jupyter Notebook

Day 5 — Matplotlib & Week 1 Mini-Notebook — 8 hours

Learning Objectives

- Create labeled line, scatter, bar, and histogram plots.
- Combine multiple plots into one figure using subplots.
- Assemble a complete mini-notebook using NumPy, Pandas, and Matplotlib together.

Key Topics

- Why visualization matters for exploration and communication
- Matplotlib basics: plot, labels, title, show
- The four core plot types: line, scatter, bar, histogram
- Subplots for side-by-side comparison
- Assembling a load → process → visualize notebook

Lesson Content

5.1 Why Visualization Matters

Numbers in a table hide patterns that a chart reveals instantly — trends, clusters, outliers, and relationships between variables. Visualization is both an exploration tool (understanding data during analysis) and a communication tool (explaining findings to others), and it is the foundation of the Exploratory Data Analysis (EDA) work in Week 2.

5.2 The Matplotlib Basics

Matplotlib is the core plotting library. The standard workflow: import pyplot, call a plotting function, add labels and a title, then show. Every plot should always be labeled — an unlabeled chart is not a finding, it's a puzzle.

```
import matplotlib.pyplot as plt
plt.plot(x, y)
plt.xlabel("Time")
```



```
plt.ylabel("Value")
plt.title("Value over Time")
plt.show()
```

5.3 The Four Core Plot Types

Plot	Function	Use It To Show
Line	<code>plt.plot()</code>	A trend or change over a continuous axis (e.g. time)
Scatter	<code>plt.scatter()</code>	The relationship between two numeric variables
Bar	<code>plt.bar()</code>	Comparison of a value across categories
Histogram	<code>plt.hist()</code>	The distribution (shape/spread) of a single variable

5.4 Subplots

Subplots place multiple charts in one figure for side-by-side comparison, using `plt.subplots()` to create a grid of axes. This is essential for EDA, where you often want to see several variables' distributions at once.

```
fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].hist(df["age"])
axes[0].set_title("Age distribution")
axes[1].scatter(df["age"], df["income"])
axes[1].set_title("Age vs. Income")
plt.tight_layout()
plt.show()
```

5.5 Bringing It Together

Week 1 closes with a single mini-notebook that uses all four libraries in sequence: NumPy for a computation, Pandas to load and clean a dataset, and Matplotlib to visualize it — the exact pipeline shape (load → process → visualize) that every project in this program follows.

Hands-On Lab: Week 1 Integrated Mini-Notebook

- Step 1: Load and clean a provided dataset with Pandas (handling any missing values).
- Step 2: Use NumPy to compute at least one derived numeric feature or summary statistic.
- Step 3: Produce at least three labeled plots (including a histogram and a scatter plot) exploring the data.
- Step 4: Write Markdown cells explaining what each visualization reveals about the data.
- Step 5: Commit the finished notebook and its requirements.txt to GitHub with a clear commit message.

Tools Used

Matplotlib • Pandas • NumPy • Jupyter Notebook • Git & GitHub

5. Week 1 Deliverables

By the end of Week 1, every intern must submit the following to their mentor and GitHub repository:

- A reproducible Python environment with a committed requirements.txt.
- A Python fundamentals notebook (functions, list comprehensions, a small class) with Markdown documentation.
- A NumPy notebook demonstrating array creation, slicing, boolean masking, and broadcasting.
- A Pandas notebook loading, cleaning, filtering, and aggregating a real dataset.
- The Week 1 integrated mini-notebook (NumPy + Pandas + Matplotlib) with at least three labeled plots.
- All Week 1 notebooks committed to the intern's GitHub repository with clear commit messages.

6. Week 1 Evaluation Criteria

Scored by the assigned mentor at the end of Week 1 using the criteria below (extracted from the program's 100-point internal rubric).

Criterion	50-69 Developing	70-84 Proficient	85-100 Excellent
Python code quality & notebook structure	Basic structure, some comments	Clean, readable, well-documented	Production-grade, fully reproducible
NumPy / Pandas proficiency	Uses libraries with guidance	Manipulates arrays and DataFrames independently	Fluent; vectorizes and cleans data efficiently
Data cleaning & correctness	Basic cleaning, some issues missed	Handles missing values and types correctly	Thorough, justified cleaning decisions
Visualization quality	Unlabeled or unclear plots	Clear, labeled, appropriate plot types	Insightful plots that reveal real patterns
Git workflow & documentation	Sporadic commits, weak notebook narrative	Regular commits, clear Markdown documentation	Consistent, descriptive, well-organized
Attendance & punctuality	3-6 absences	1-2 absences, on time	Perfect attendance, proactive

7. Technical Stack — Week 1

Language	Python 3.10+
Environment	venv / conda, pip, requirements.txt
Numerical Computing	NumPy
Tabular Data	Pandas
Visualization	Matplotlib
Workflow	Jupyter Notebook, VS Code, Git & GitHub, Google Colab

8. Notes & Best Practices

Every notebook this week must read like a short report: Markdown cells explaining each step, clean and documented code, and visible, reproducible outputs. Fix a random seed wherever randomness is used so results are repeatable — this reproducibility habit, established in Week 1, is required for every deliverable through to the Phase 3 capstone project.